

PRAKTIKUM 13

MEDIAPIPE HAND

Nama : Kayla Zafir Abisheka

NIM : 234308075

Kelas : TKA-6C

Akun Github (Tautan) : <https://github.com/kaylazafirabisheka>

Abstrak

Praktikum ini mengimplementasikan sistem deteksi dan pelacakan tangan secara real-time menggunakan MediaPipe yang diintegrasikan dengan OpenCV menggunakan bahasa pemrograman Python. Sistem memanfaatkan webcam sebagai sumber input untuk menangkap frame video, kemudian memproses citra tersebut guna mendeteksi keberadaan tangan serta mengidentifikasi 21 titik landmark yang merepresentasikan struktur anatomi tangan. Landmark yang terdeteksi divisualisasikan pada layar beserta garis penghubungnya sehingga struktur tangan dapat diamati secara langsung.

Pengembangan program dilakukan melalui beberapa tahapan, yaitu deteksi keberadaan tangan, visualisasi landmark, klasifikasi tangan kanan dan kiri, serta identifikasi orientasi telapak dan punggung tangan. Klasifikasi tangan dilakukan menggunakan fitur handedness yang disediakan oleh MediaPipe, sedangkan penentuan orientasi tangan dilakukan melalui analisis hubungan geometris antar koordinat landmark.

Hasil percobaan menunjukkan bahwa sistem mampu mendeteksi dan melacak tangan dengan performa yang cukup stabil pada kondisi pencahayaan yang memadai. Secara keseluruhan, implementasi ini membuktikan bahwa teknik computer vision berbasis landmark efektif digunakan untuk pelacakan tangan secara real-time dan dapat menjadi dasar pengembangan sistem pengenalan gestur serta aplikasi interaksi tanpa sentuhan yang lebih kompleks.

1. Pendahuluan :

Perkembangan *computer vision* memungkinkan komputer mengenali objek dari citra atau video secara real-time. Salah satu penerapannya adalah pendeteksian tangan (*hand tracking*), yang banyak digunakan pada sistem interaksi tanpa sentuhan, pengenalan gestur, dan aplikasi berbasis kamera lainnya. Pada praktikum ini digunakan MediaPipe Hands, sebuah framework dari Google yang mampu mendeteksi titik-titik landmark tangan secara akurat.

MediaPipe dipadukan dengan OpenCV untuk mengambil citra dari kamera dan menampilkan hasil deteksi secara langsung. Sistem ini bekerja dengan menangkap gambar dari kamera, memprosesnya, lalu menampilkan titik-titik tangan beserta garis penghubungnya pada layar. Melalui praktikum ini, mahasiswa dapat memahami alur dasar pendeteksian tangan berbasis citra digital, mulai dari pengambilan gambar hingga visualisasi landmark tangan secara real-time menggunakan Python.

2. Tujuan Praktikum :

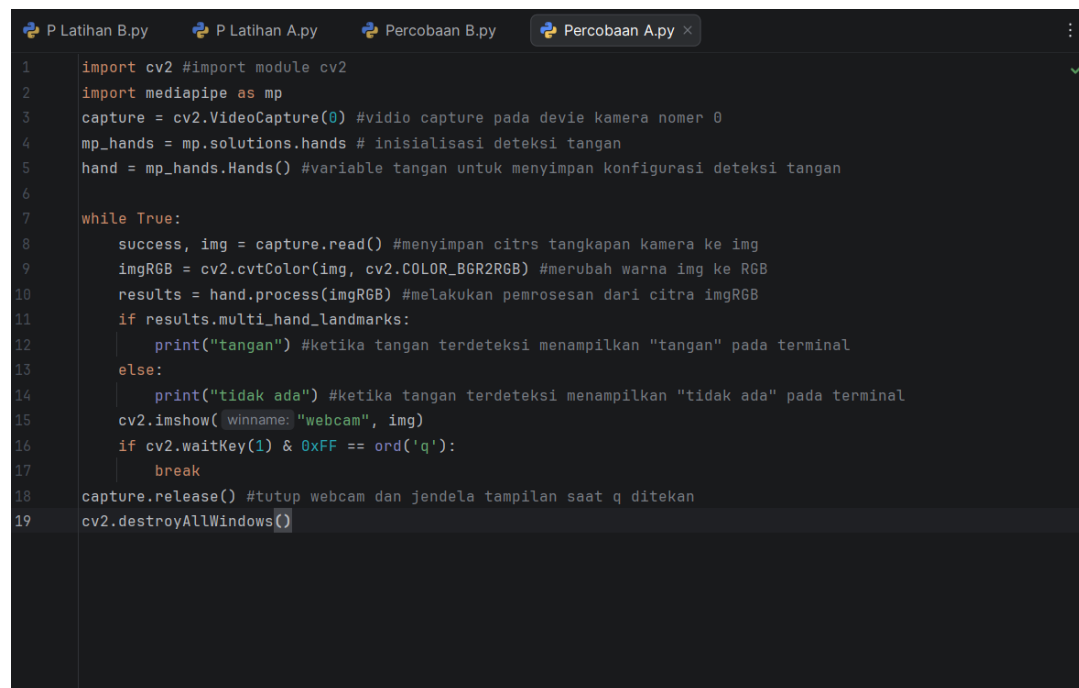
- a. Mengimplementasikan pendeteksian tangan secara real-time menggunakan kamera.
- b. Mempelajari penggunaan MediaPipe Hands untuk mendeteksi landmark tangan.
- c. Menggunakan OpenCV sebagai media pengambilan dan penampilan citra.
- d. Memahami alur program hand tracking dari proses input hingga visualisasi.
- e. Menampilkan titik landmark tangan dan koneksinya pada layar.

3. Manfaat Praktikum

- Memahami dasar penerapan *computer vision* pada pendeteksian tangan.
- Menambah keterampilan penggunaan MediaPipe dan OpenCV dalam Python.
- Menjadi dasar pengembangan sistem pengenalan gestur tangan.
- Memberikan gambaran penerapan pengolahan citra secara real-time.
- Dapat dikembangkan menjadi sistem kontrol berbasis gerakan tangan.

4. Hasil Program

A. Deteksi Tampilan Tangan



```
1 import cv2 #import module cv2
2 import mediapipe as mp
3 capture = cv2.VideoCapture(0) #vidio capture pada devie kamera nomer 0
4 mp_hands = mp.solutions.hands # inisialisasi deteksi tangan
5 hand = mp_hands.Hands() #variable tangan untuk menyimpan konfigurasi deteksi tangan
6
7 while True:
8     success, img = capture.read() #menyimpan citrs tangkapan kamera ke img
9     imgRGB = cv2.cvtColor(img, cv2.COLOR_BGR2RGB) #merubah warna img ke RGB
10    results = hand.process(imgRGB) #melakukan pemrosesan dari citra imgRGB
11    if results.multi_hand_landmarks:
12        print("tangan") #ketika tangan terdeteksi menampilkan "tangan" pada terminal
13    else:
14        print("tidak ada") #ketika tangan terdeteksi menampilkan "tidak ada" pada terminal
15        cv2.imshow( winname: "webcam", img)
16        if cv2.waitKey(1) & 0xFF == ord('q'):
17            break
18 capture.release() #tutup webcam dan jendela tampilan saat q ditekan
19 cv2.destroyAllWindows()
```

B. Hands Landmarks

```
P Latihan B.py P Latihan A.py Percobaan B.py x Percobaan A.py
1 import cv2 #import module opencv
2 import mediapipe
3 capture = cv2.VideoCapture(0)
4 mediapipehand = mediapipe.solutions.hands
5 tangan = mediapipehand.Hands(max_num_hands=1)
6 mpdraw = mediapipe.solutions.drawing_utils
7
8 while True:
9     success, img = capture.read() # Read video frame by frame
10    imgRGB = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
11    result = tangan.process(imgRGB)
12    if result.multi_hand_landmarks:
13        for titiktangan in result.multi_hand_landmarks:
14            mpdraw.draw_landmarks(img, titiktangan, mediapipehand.HAND_CONNECTIONS)
15            for id, titik in enumerate(titiktangan.landmark):
16                print(id)
17                print(titik.x)
18                print(titik.y)
19    cv2.imshow( winname: "webcam",img)
20    cv2.waitKey(10)
21    if cv2.waitKey(10) & 0xFF == ord('q'):
22        break #Tutup webcam dan jendela tampilan saat 1 ditekan
23 cap.release()
24 cv2.destroyAllWindows()
25
```

Latihan

A. Buatlah program deteksi posisi tangan kanan dan kiri

```
P Latihan B.py Percobaan B.py P Latihan A.py x Percobaan A.py
1 > import ...
3
4 capture = cv2.VideoCapture(0)
5
6 mp_hands = mp.solutions.hands
7 tangan = mp_hands.Hands(max_num_hands=2)
8 mpdraw = mp.solutions.drawing_utils
9
10 while True:
11     success, frame = capture.read()
12     if not success:
13         break
14
15     imgRGB = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
16     results = tangan.process(imgRGB)
17
18     if results.multi_hand_landmarks:
19         for idx, titik tangan in enumerate(results.multi_hand_landmarks):
20             mpdraw.draw_landmarks(
21                 frame, titik tangan, mp_hands.HAND_CONNECTIONS
22             )
23
24             #ambil titik pergelangan (landmarks 0) untuk posisi teks
25             h, w, c = frame.shape
26             x = int(titik tangan.landmark[0].x*w)
27             y = int(titik tangan.landmark[0].y*h)
28
29             # klasifikasi kiri / kanan
30             hand = results.multi_handedness[idx]
31             label = hand.classification[0].label # "left"/"right"
```

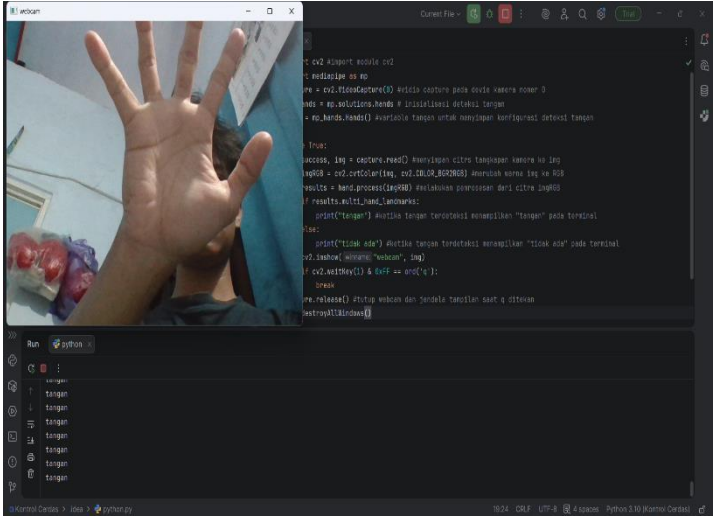
```
P Latihan B.py Percobaan B.py P Latihan A.py x Percobaan A.py
20 mpdraw.draw_landmarks(
21     frame, titiktangan, mp_hands.HAND_CONNECTIONS
22 )
23
24 #ambil titik pergelangan (landmarks 0) untuk posisi teks
25 h, w, c = frame.shape
26 x = int(titiktangan.landmark[0].x*w)
27 y = int(titiktangan.landmark[0].y*h)
28
29 # klasifikasi kiri / kanan
30 hand = results.multi_handedness[idx]
31 label = hand.classification[0].label # "left"/"right"
32
33 if label == "Right":
34     cv2.putText(frame, text: "Kiri", org: (x, y - 20),
35                 cv2.FONT_HERSHEY_PLAIN, fontScale: 3, color: (255, 0, 0), thickness: 3)
36 else:
37     cv2.putText(frame, text: "Kanan", org: (x, y - 20),
38                 cv2.FONT_HERSHEY_PLAIN, fontScale: 3, color: (0, 0, 255), thickness: 3)
39
40 cv2.imshow( winname: "webcam", frame)
41
42 if cv2.waitKey(1) & 0xFF == ord('q'):
43     break
44
45 capture.release()
46 cv2.destroyAllWindows()
```

B. Buatlah program deteksi posisi tangan depan dan belakang menggunakan posisi landmarks tangan.

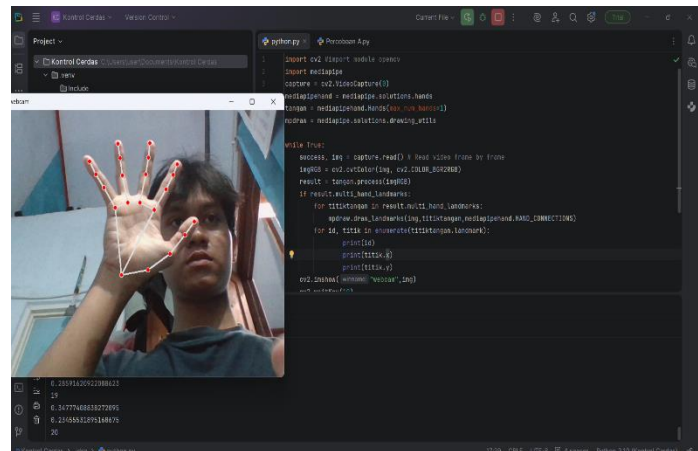
```
P Latihan B.py Percobaan B.py x P Latihan A.py Percobaan A.py
1 import cv2 #import module opencv
2 import mediapipe
3 capture = cv2.VideoCapture(0)
4 mediapipehand = mediapipe.solutions.hands
5 tangan = mediapipehand.Hands(max_num_hands=1)
6 mpdraw = mediapipe.solutions.drawing_utils
7
8 while True:
9     success, img = capture.read() # Read video frame by frame
10    imgRGB = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
11    result = tangan.process(imgRGB)
12    if result.multi_hand_landmarks:
13        for titiktangan in result.multi_hand_landmarks:
14            mpdraw.draw_landmarks(img,titiktangan,mediapipehand.HAND_CONNECTIONS)
15            for id, titik in enumerate(titiktangan.landmark):
16                print(id)
17                print(titik.x)
18                print(titik.y)
19    cv2.imshow( winname: "webcam",img)
20    cv2.waitKey(10)
21    if cv2.waitKey(10) & 0xff == ord('q'):
22        break #Tutup webcam dan jendela tampilan saat 1 ditekan
23 cap.release()
24 cv2.destroyAllWindows()
25
```

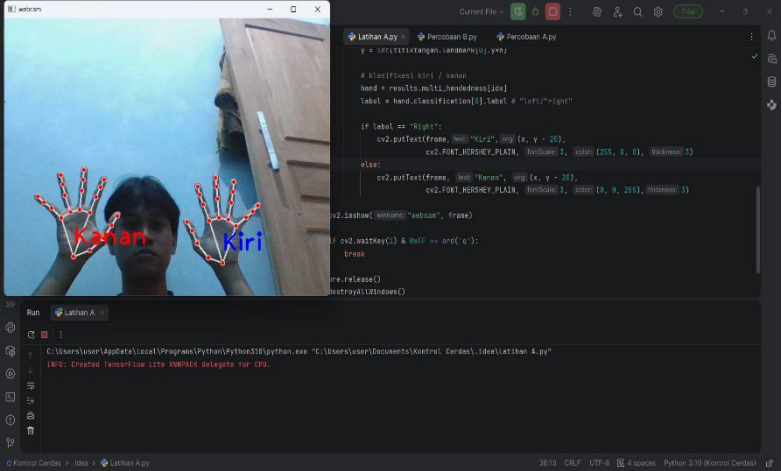
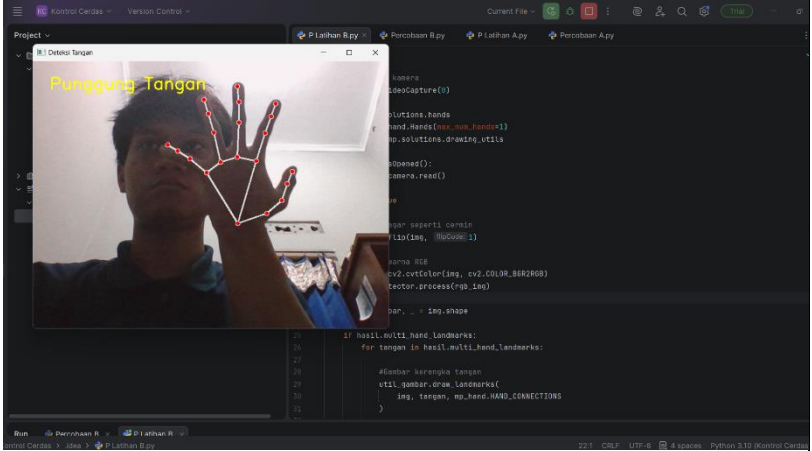
Data dan Output Hasil Pengamatan :

- Hasil pengamatan berdasarkan dengan dataset baru.

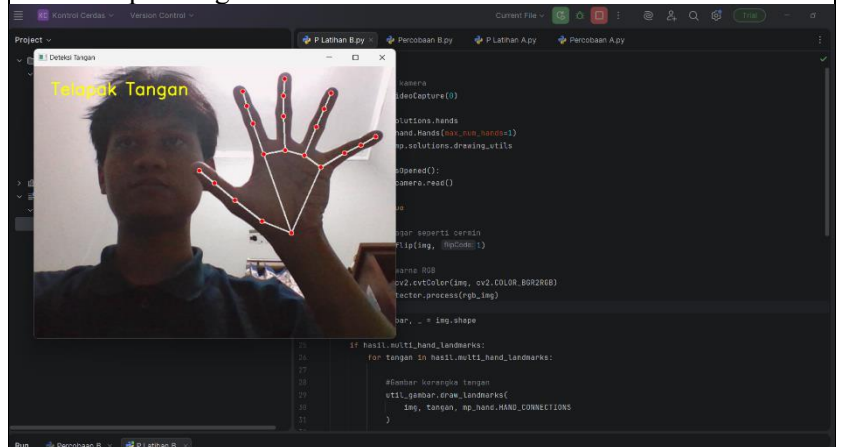
No	Variabel	Hasil Pengamatan
1.	Deteksi Tampilan Tangan	

2. Hands Landmarks



3.	Deteksi posisi tangan kanan dan kiri	 The screenshot shows a video feed of a person's hands held up. Red dots mark the joints of both hands, and lines connect them to form a skeletal model. The right hand is labeled 'Kanan' in red text, and the left hand is labeled 'Kiri' in blue text. The application is running in a PyCharm IDE with a Python script visible in the background.
4.	Deteksi posisi tangan depan dan belakang menggunakan posisi landmarks tangan.	- Punggung tangan  The screenshot shows a video feed of a person's back of the hand held up. Red dots mark the joints, and lines connect them. The label 'Punggung Tangan' is written in yellow text above the hand. The application is running in a PyCharm IDE with a Python script visible in the background.

- Telapak tangan



4. Analisis Program

Pada praktikum ini dilakukan implementasi sistem deteksi dan pelacakan tangan menggunakan framework MediaPipe yang diintegrasikan dengan library OpenCV. Sistem memanfaatkan kamera sebagai sumber input citra melalui fungsi `cv2.VideoCapture(0)`, kemudian setiap frame yang ditangkap diproses secara real-time untuk mendeteksi keberadaan tangan beserta struktur landmark-nya. Proses konversi warna dari BGR ke RGB menggunakan `cv2.cvtColor()` dilakukan agar format citra sesuai dengan kebutuhan pemrosesan MediaPipe.

Pada bagian Deteksi Tampilan Tangan, program difokuskan untuk mengidentifikasi ada atau tidaknya tangan pada frame kamera. Fungsi `hands.process(imgRGB)` digunakan untuk memproses citra yang telah dikonversi. Hasil pemrosesan disimpan dalam variabel `results` yang kemudian diperiksa menggunakan kondisi `if results.multi_hand_landmarks:`. Jika kondisi bernilai benar, maka sistem menyatakan bahwa tangan terdeteksi. Sebaliknya, jika tidak ada landmark yang terbaca, maka sistem menyimpulkan tidak ada tangan pada frame tersebut. Tahap ini menunjukkan mekanisme dasar deteksi objek berbasis model machine learning yang telah dilatih sebelumnya.

Pada bagian Hands Landmarks, sistem tidak hanya mendeteksi keberadaan tangan, tetapi juga menampilkan 21 titik landmark yang merepresentasikan struktur anatomi tangan. Fungsi `mpdraw.draw_landmarks()` digunakan untuk menggambar titik-titik landmark serta garis koneksinya pada citra asli. Landmark yang dihasilkan memiliki koordinat ter-normalisasi (x, y, z), di mana nilai x dan y menunjukkan posisi relatif terhadap ukuran frame, sedangkan nilai z menunjukkan kedalaman relatif terhadap kamera. Visualisasi ini membuktikan bahwa sistem mampu melakukan pelacakan struktur tangan secara detail dan real-time.

Pada percobaan Deteksi Posisi Tangan Kanan dan Kiri, sistem memanfaatkan atribut `results.multi_handedness` yang disediakan oleh MediaPipe untuk mengklasifikasikan jenis tangan yang terdeteksi. Informasi label “Left” atau “Right” diambil dari hasil klasifikasi tersebut, kemudian ditampilkan pada layar menggunakan fungsi `cv2.putText()`.

Proses ini menunjukkan bahwa metadata hasil deteksi dapat dimanfaatkan untuk memberikan informasi tambahan mengenai jenis tangan tanpa perlu membuat model klasifikasi baru.

Selanjutnya, pada percobaan Deteksi Posisi Tangan Depan dan Belakang, analisis dilakukan berdasarkan hubungan geometris beberapa landmark utama, seperti pergelangan tangan, pangkal jari telunjuk, dan pangkal jari kelingking. Dengan membandingkan posisi relatif titik-titik tersebut, sistem dapat menentukan apakah tangan berada dalam posisi telapak menghadap kamera atau punggung tangan menghadap kamera. Logika pengambilan keputusan ini berbasis perhitungan sederhana terhadap koordinat landmark, sehingga menunjukkan bahwa data landmark tidak hanya berfungsi untuk visualisasi, tetapi juga dapat dimanfaatkan untuk analisis spasial yang lebih lanjut.

Secara keseluruhan, program yang dibuat telah mampu melakukan deteksi tangan, pelacakan landmark, klasifikasi tangan kiri dan kanan, serta identifikasi orientasi telapak tangan secara real-time dengan performa yang cukup stabil pada kondisi pencahayaan yang memadai.

5. Kesimpulan

Berdasarkan percobaan dan analisis yang telah dilakukan, dapat disimpulkan bahwa:

1. Sistem berbasis MediaPipe dan OpenCV mampu melakukan deteksi dan pelacakan tangan secara real-time dengan cukup stabil menggunakan 21 titik landmark.
2. Program berhasil mendeteksi keberadaan tangan pada frame kamera melalui pengecekan `multi_hand_landmarks`, sehingga dapat membedakan kondisi ada atau tidaknya tangan di depan kamera.
3. Visualisasi landmark dan koneksi antar titik berhasil ditampilkan dengan baik, sehingga struktur anatomi tangan dapat diamati secara jelas pada layar.
4. Sistem mampu mengklasifikasikan tangan kanan dan kiri menggunakan fitur `handedness` yang disediakan oleh MediaPipe tanpa perlu model tambahan.
5. Data koordinat landmark dapat dimanfaatkan untuk analisis lebih lanjut, seperti menentukan orientasi telapak dan punggung tangan berdasarkan hubungan

geometris antar titik.

6. Secara keseluruhan, program telah memenuhi tujuan praktikum dan dapat menjadi dasar pengembangan sistem pengenalan gestur tangan yang lebih kompleks.

6. Saran

Untuk pengembangan selanjutnya, sistem sebaiknya diuji pada berbagai kondisi pencahayaan, jarak kamera, dan latar belakang yang lebih kompleks guna mengetahui batas performa deteksi secara menyeluruh. Selain itu, logika penentuan orientasi tangan dapat ditingkatkan dengan mempertimbangkan lebih banyak kombinasi landmark agar akurasi klasifikasi lebih stabil. Pengembangan lebih lanjut juga dapat diarahkan pada pembuatan sistem pengenalan gestur yang lebih kompleks, seperti kontrol presentasi, pengendalian objek virtual, atau integrasi dengan sistem robotik sehingga hasil praktikum memiliki nilai aplikatif yang lebih luas.

2. Daftar Pustaka :

1. Bradski, G., & Kaehler, A. (2017). Learning OpenCV 3: Computer Vision in C++ with the OpenCV Library. O'Reilly Media.
2. Lugaresi, C., Tang, J., Nash, H., et al. (2020). MediaPipe: A Framework for Building Perception Pipelines. Google AI.
3. Zhang, Z., et al. (2020). Hand Gesture Recognition and Human–Computer Interaction: A Review. IEEE Access, 8, 74531–74549.
4. Google Developers. (n.d.). MediaPipe Hands. Diakses dari https://developers.google.com/mediapipe/solutions/vision/hand_landmarker
5. OpenCV.org. (n.d.). OpenCV Documentation. Diakses dari <https://docs.opencv.org>